2025 28th International Conference on Computer and Information Technology (ICCIT)
19-21 December 2025, Cox's Bazar, Bangladesh

An Efficient Distributed Framework for Top-k Dominating Query Over Uncertain Databases

Md. Safwan Zaher Asif
Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna-9203, Bangladesh
safwanzaher123@gmail.com

K. M. Azharul Hasan
Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna-9203, Bangladesh
az@cse.kuet.ac.bd

2025 28th International Conference on Computer and Information Technology (ICCIT) | 979-8-3315-7867-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCIT68739.2025.11490166

***Abstract*—Probabilistic top-k dominating (PTD) queries are essential for retrieving the most influential objects from uncertain data, but executing them over large datasets is computationally expensive. This work presents a distributed framework that evaluates PTD queries in a Hadoop environment by combining a filtering stage with a lightweight refinement step. The framework introduces a dynamic candidate pruning strategy that exploits lower and upper bounds to eliminate weak objects early, and an aggregated emission strategy that reduces redundant intermediate data during MapReduce execution. These methods improve query efficiency while preserving exact results. The proposed framework was tested on two datasets, a synthetic smartphone collection and a Bangladesh road dataset derived from OSM geometries, and the results show that the wall clock time was reduced by 34.2% on the synthetic dataset and 24.7% on the real dataset, while the communication cost remained unchanged. Communication cost remained unchanged as output cardinality was kept identical to the baseline for fair comparison. These improvements confirm that the framework works effectively across different uncertain domains, demonstrating that pruning and emission aggregation can cut unnecessary computation and data transfer and making the approach practical for large-scale probabilistic query processing in distributed environments.**

***Index Terms*—Probabilistic top-k dominating query, Uncertain database, Distributed query processing, Hadoop MapReduce, Dynamic dominance region, Minimum bounding rectangle**

I. Introduction

Probabilistic databases are increasingly important in modern applications where data is uncertain, incomplete, or derived from complex sources such as sensor networks [1], spatial data systems [2], and recommendation systems [3]. In many real-world scenarios, data uncertainty arises naturally due to measurement errors, network transmission delays. For example, GPS-based location tracking systems often produce imprecise coordinates due to signal interceptions, while IoT sensor networks generate readings with varying degrees of accuracy depending on environmental conditions [4].

Among the types of query in probabilistic data, the Probabilistic Top-k Dominating (PTD) query plays a significant role in decision-making applications [5] [6] [7]. PTD queries retrieve the top-k objects that dominate others with the highest probabilities, making them particularly valuable for multi-criteria decision support systems. Unlike traditional skyline queries [8] that return all non-dominated points, or simple top-k queries that require user-defined scoring functions, PTD queries provide a better result size while considering the probabilistic nature of uncertain data. This combination makes them highly suitable for applications such as route planning in transportation networks, where road conditions are uncertain, financial portfolio selection, where market conditions exhibit probabilistic behavior, and many other applications.

Modern applications generate massive amounts of uncertain data that cannot be efficiently processed on a single machine. It requires a distributed environment. However, existing PTD algorithms face significant challenges when deployed in distributed environments such as Hadoop MapReduce [9]. The primary bottlenecks arise from excessive candidate processing, where non-contributing objects consume computational resources, and high communication costs due to large intermediate data transfers between distributed nodes [10]. We developed a Distributed Top-k Dominating query algorithm that demonstrates the following contributions:

- Development of Dynamic Smart Candidate Pruning (DSCP), an early filtering mechanism that eliminates non-contributing candidate instances during the filtering phase, reducing unnecessary processing overhead.
- Introduction of Aggregated Emission Strategy (AES), which minimizes shuffle and communication costs by consolidating multiple emissions into compact aggregated records.
- Experimental evaluation of synthetic and real-world datasets that demonstrate significant performance improvements while maintaining query accuracy.
- Validation of the scalability and effectiveness of the proposed framework in distributed environments, achieving a 24–34% reduction in wall-clock time compared to baseline approaches.

II. Related Work

Research on top-k dominating queries spans three main areas: centralized processing on certain data, uncertain data handling, and distributed query processing frameworks.

Top-k Dominating Queries on Certain Data: The foundational work by Papadias et al. [11] introduced Top-k Dominating queries as skyline variants that return k objects dominating the largest number of others, combining advantages of skyline and top-k queries without requiring user-

defined scoring functions. Yiu and Mamoulis [12] optimized these queries for multidimensional data using aggregate R-tree indexes to estimate score limits and allow effective pruning. Han et al. [13] developed the TDEP approach for vertically decomposed data, while recent advances include bitmap-based approaches [14] and algorithms optimized for massive data processing [15].

Probabilistic Top-k Dominating Queries: The extension to uncertain data was pioneered by Lian and Chen [5], [7], who defined PTD queries under possible-worlds semantics and proposed a pruning mechanism with indexing methods for efficient domination probability computation. Zhang et al. [16] introduced threshold-based PTD queries (PtopkQ) that retrieve objects whose domination probability exceeds specified thresholds. For dynamic scenarios, Feng et al. [17] addressed sliding window processing on uncertain streams, while Santoso and Chiu [18] proposed close dominance graph frameworks for continuous queries. Recent work by Lai et al. [4], [19] developed edge computing solutions and pruning schemes for IoT data streams.

Distributed Query Processing: Early probabilistic database systems include BayesStore [3] to manage uncertain repositories with graphical models, ULDB [20] for handling uncertainty and lineage, and MayBMS [21] using world-set decompositions. However, these systems assumed centralized architectures. For distributed dominance queries, Li et al. [22] studied distributed top-k queries on probabilistic data, while Park et al. [23] presented MapReduce-based methods for skyline and probabilistic skyline computation. Zhang et al. [24] and Zhou et al. [25] further advanced parallel skyline evaluation techniques.

The gap in distributed PTD processing was addressed by Rai and Lian [10], who proposed the first MapReduce-based distributed PTD framework with cost-model-driven index distribution and pruning at the mapper and reducer stages. However, their approach still suffers from emission overhead and limited candidate pruning efficiency, particularly for large-scale datasets, where communication costs become the primary performance bottleneck.

III. Preliminaries and Problem Definition

A. Certain and Uncertain Data

In traditional databases, attributes have single exact values, so queries have no ambiguity. Modern applications, however, often produce *uncertain* data, where attributes are ranges or probability distributions. A distributed uncertain database extends the uncertain database model in which uncertain objects are spread across multiple servers connected in a network.

Consider an uncertain database of cameras whose instances combine price and quality with appearance probabilities in Table I. For example, "Canon1 (750, 9)" denotes a $750 camera with quality 9 that appears with probability 0.60.

B. Top-k Dominating Query

The top-k dominating queries identify objects that dominate others on the basis of multiple criteria. An object dominates another when it outperforms in all dimensions [11]. Given two instances x_i and y_j from uncertain objects X and Y, respectively, and a query point q, we say that x_i dynamically dominates y_j with respect to q, denoted as $x_i \prec_q y_j$, if:

$$|x_i.A_k - q.A_k| \leq |y_j.A_k - q.A_k|, \quad \forall k \in [1, d], \tag{1}$$

$$|x_i.A_k - q.A_k| < |y_j.A_k - q.A_k|, \quad \text{for some } k \in [1, d] \tag{2}$$

Dynamic dominance helps rank uncertain instances relative to a query point, which is crucial to selecting the top-k results.

TABLE I
Example of Uncertain Data Objects

Object	Instance	Price	Quality	Probability
Canon	Canon1	750	9	0.6
	Canon2	700	7	0.4
Nikon	Nikon1	680	8	0.5
	Nikon2	720	9	0.5

C. Hadoop MapReduce Framework

MapReduce is a distributed programming model for processing large datasets across many servers; Hadoop implements it via HDFS for storage and a MapReduce engine for execution. For PTD queries, data in HDFS are first partitioned, so each split contains uncertain objects with their instances and appearance probabilities. During the Mapper phase, each mapper applies Dynamic Dominance Region tests to compute instance-level domination and derive local lower/upper bounds (LB/UB). Then the shuffle phase. Finally, in the Reducer phase, LB/UB values are aggregated per object and candidates are filtered to produce the global top-k.

D. Dynamic Dominance Region

The Dynamic Dominance Region (DDR) captures the area within multi-dimensional space where an instance dynamically dominates other instances relative to a specific query point q. It represents the set of points for which a given instance remains most favorable compared to others. The DDR consists of four symmetric regions around the query point: top-right (TR), top-left (TL), bottom-left (BL), and bottom-right (BR).

Figure 1 illustrates the DDR concept using camera instances from Table I. Any instance falling within these regions is considered dominated with respect to the query point.

E. Lower Bound and Upper Bound

To efficiently approximate domination probabilities, the lower bound (LB) and Upper Bound (UB) are defined at both the instance and object levels. The Minimum Bounding Rectangle (MBR) is the smallest rectangle that fully encloses a set of uncertain object instances [10], defined by extreme coordinate values: (x_{min}, x_{max}) and y-axis (y_{min}, y_{max}) values.

If an MBR lies fully inside the DDR of an instance, the object is fully dominated; if it overlaps partially, it is partially dominated. Figure 2 illustrates the MBR concept with partial dominance from the values of Table I. At the instance level, the LB of an instance is computed by summing its own probability

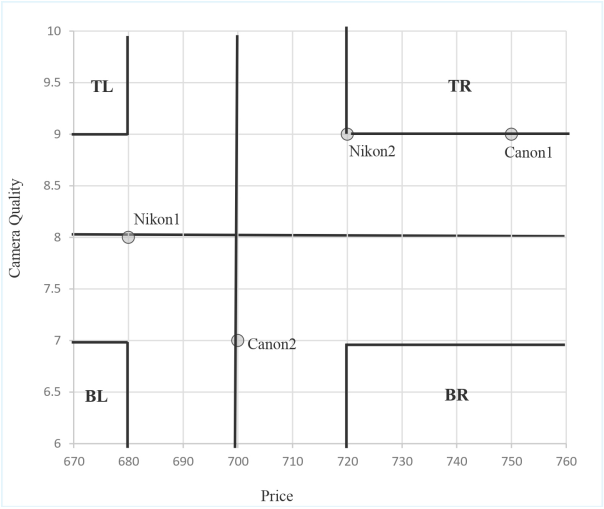

Fig. 1. Dynamic Dominance Region (DDR) visualization showing four quadrants (TL, TR, BL, BR) around query point. Instances within these regions are dominated.

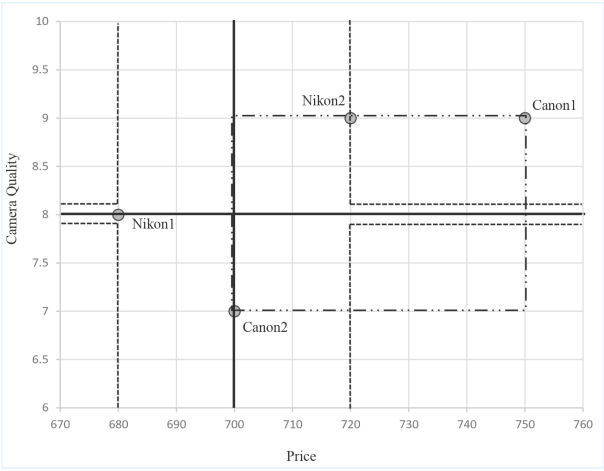

Fig. 2. Minimum Bounding Rectangle (MBR) visualization of an object which is partially dominated by the DDR of an instance.

with those of all instances from fully dominated objects, plus dominated local instances within the same partition:

$$LB(o_i) = \sum_{o_j \in DDR(o_i)} I_{fully} + \sum_{k \in DDR(o_i)} P(o_k), \quad (3)$$

where I_{fully} represents the indicator function for fully contained MBRs and $P(o_k)$ denotes the probability of objects within the DDR. The UB provides a more optimistic estimate by extending the lower bound calculation to include probabilities of partially dominated objects:

$$UB(o_i) = \sum_{o_j \in DDR(o_i)} I_{fully} + \sum_{o_j \in DDR(o_i)} I_{partial} + \sum_{k \in DDR(o_i)} P(o_k), \quad (4)$$

where $I_{partial}$ accounts for partially contained MBRs that intersect but are not completely contained within the DDR.

$$LB(O) = \sum_{t \in O} LB(t), \quad UB(O) = \sum_{t \in O} UB(t). \quad (5)$$

At the object level, the bounds are obtained by aggregating the LB and UB values across all instances belonging to the same object.

IV. Proposed Distributed PTD Framework

We propose a novel workflow that integrates *Dynamic Smart Candidate Pruning (DSCP)* and an *Aggregated Emission Strategy (AES)* into the MapReduce pipeline. Unlike [10], which incurs high emission overhead and employs relatively weak pruning, our framework achieves 24–34% runtime reduction while preserving exact PTD semantics. The baseline framework [10], although effective in handling large uncertain datasets, suffers from two major drawbacks. First, *redundant shuffle emissions*: each instance repeatedly emits competitor identifiers during filtering, leading to excessive intermediate data and high communication cost. Second, *weak pruning*: the candidate elimination strategy is not aggressive, causing many weak objects to survive into refinement, which increases wall-clock time. These limitations motivate our proposed extensions. The workflow of the proposed framework is illustrated in Figure 3.

A. Workflow Overview

The framework retains the two-phase structure of filtering and refinement but enhances both phases with DSCP and AES. The overall workflow proceeds as follows:

FilteringMapper: Each mapper partitions uncertain objects and computes local lower bound (LB) and upper bound (UB) scores based on DDR–MBR overlaps. DSCP is applied to aggressively prune weak candidates early by comparing each object's UB against the k-th largest LB (τ). Only strong candidates survive.

FilteringReducer: Reducers aggregate LB and UB values of surviving candidates across partitions. AES is applied here to reduce redundant emissions by merging competitor identifiers into aggregated sets, thereby shrinking shuffle size.

RefinementMapper: A map-only job finalizes the ranking. Candidate objects are re-scored against global dominance relations using the aggregated information from AES. The top-k dominating objects are then produced.

This workflow ensures that pruning is performed earlier and more effectively (via DSCP) while emissions are minimized (via AES), leading to lower wall-clock time but the same communication cost as baseline [10] to preserve correctness.

B. Dynamic Smart Candidate Pruning (DSCP)

The DSCP strategy refines the pruning mechanism by comparing the UB of each object with the k-th largest LB (τ) among current candidates. If $UB(o) < \tau$, the object o is pruned immediately. DSCP recalculates τ dynamically as new candidates are added, ensuring that only competitive objects

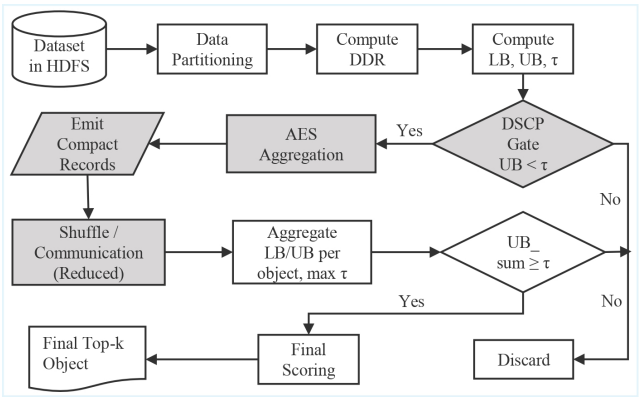

Fig. 3. Workflow of the proposed distributed PTD framework integrating DSCP and AES.

are carried forward. This significantly reduces the number of weak candidates that survive into refinement, cutting unnecessary computation. But in the baseline, after computing lower bound (LB) and upper bound (UB) for each object instance, all instances with $UB(o) \geq \tau$ are forwarded to the reducer. This "emit-all-that-pass" strategy leads to significant redundancy.

C. Aggregated Emission Strategy (AES)

The AES replaces this with aggregated competitor sets: for each surviving instance, competitor identifiers are combined into a single merged set before emission. This reduces the total number of intermediate records, lowering shuffle overhead while preserving correctness. For example, if one instance competes with 10 other objects, it emits 10 records. Across thousands of instances, this results in tens of millions of shuffle records. The cost is high because each record consumes network bandwidth. Also the Reducer workload increases with record count. AES restructures emissions by aggregating multiple competitors into one compact record per instance. The new emission format due to AES is:

$$\langle \texttt{objectId},\ \texttt{instanceId},\ \texttt{eid}_1;\ldots;\texttt{eid}_N,\ lb,\ ub,\ \tau \rangle$$

Here, all competitors, *(eid1 … eidN)* are grouped together, alongside the LB, UB, and (τ) values. By using AES practical communication load decreases dramatically.

D. Algorithm Explanation

Algorithm 1 shows the whole FilteringMapper as the work mainly improved this part. The mapper ingests records $r = (o, a, pr)$ with query q, partitions N, and top-k. Each record is hashed to a partition and stored in $B[p]$. For each instance i, we set $(LB_i, UB_i) \leftarrow (i.pr, i.pr)$ and update them via DDR tests against local instances j; these instance bounds accumulate to object-level *ObjectLB*$[p][o]$ and *ObjectUB*$[p][o]$. We define τ_p as the k-th largest *ObjectLB*$[p][o]$ in partition p. After τ_p is computed, *dynamic smart candidate pruning (DSCP)* eliminates any object with *ObjectUB*$[p][o] < \tau_p$, since such an object cannot surpass the current lower-bound frontier of the top-k. For objects that survive, we construct a competitor set EIDset by collecting all other object IDs in the same partition that may still interact in dominance evaluation. At emission time, the *aggregated emission strategy (AES)* outputs a single record per surviving instance, bundling bounds and competitor context into one compact tuple. This replaces many instance–competitor pairs with a single aggregated emit, reducing redundant intermediate records and shuffle cost while preserving the information needed for correct reducer-side aggregation.

Algorithm 1: FilteringMapper with DSCP and AES

Input: records $r = (o, a, pr)$; query q; N partitions; k
buffers $B[p]$; *ObjectLB*$[p][o]$, *ObjectUB*$[p][o]$
foreach r *in* `InputSplit` **do**
  $p \leftarrow 1 + (\text{hash}(o) \bmod N)$;
  append (i, o, a, pr, p) to $B[p]$;
`// Per-partition bounds & object accumulators`
foreach *partition* p **do**
  $I \leftarrow B[p]$;
  foreach *instance* $i \in I$ **do**
    $(LB_i, UB_i) \leftarrow (i.pr, i.pr)$;
    foreach $j \in I$ **do**
      if $DDR(j, i, q)$ **then**
        $UB_i \leftarrow UB_i + j.pr$;
        if $j.o = i.o$ **then**
          $LB_i \leftarrow LB_i + j.pr$
    ObjectLB$[p][i.o] \leftarrow$ *ObjectLB*$[p][i.o] + LB_i$;
    ObjectUB$[p][i.o] \leftarrow$ *ObjectUB*$[p][i.o] + UB_i$;
    tag i with (LB_i, UB_i);
`// Compute` τ_p`: k-th largest LB per partition`
foreach *partition* p **do**
  $\tau_p \leftarrow \text{kthLargest}(\{$*ObjectLB*$[p][o] : o \in p\})$;
`// DSCP + AES emission`
foreach *partition* p **do**
  foreach *object* $o \in p$ **do**
    if *ObjectUB*$[p][o] < \tau_p$ **then**
      continue
    `// DSCP`
    $EIDset \leftarrow \{\, o' \in p : o' \neq o \,\}$;
    foreach *instance* i *of* o *in* $B[p]$ **do**
      emit key $= p$;
      val $\leftarrow \langle o,\ i.id,\ \text{join}(EIDset, ';'), ... \rangle$;
    `// AES`

V. Experimental Evaluation

A. Experimental Setup

We implemented both the baseline [10] and the proposed PTD frameworks on a pseudo-distributed Hadoop cluster.

Experiments ran on a Windows 11 (64-bit) machine with an Intel Core i5-8265U (quad-core, up to 3.90 GHz), 8 GB DDR4 RAM, and a 512 GB SSD. Java (JDK 1.8) was used for MapReduce jobs; Python with Pandas/NumPy/GeoPandas supported dataset preparation. All algorithms were executed under identical hardware and software conditions.

B. Datasets

We used two datasets. The *synthetic smartphone* dataset simulates 15 market regions (budget, mid-range, premium, gaming, *etc.*). In each region, about 50 brands are generated; each brand contributes 8–20 models. Attributes are price, battery capacity, and appearance probability, normalized to sum to one per brand. Roughly 5% of brands are outliers with extreme attribute values to stress robustness. The *Bangladesh road* dataset is constructed from road geometries. Each road segment is converted to a minimum bounding rectangle (MBR); per MBR we sample 5–11 random instances (longitude, latitude) with probabilities normalized to one per segment. This yields realistic spatial uncertainty.

C. Evaluation Metrics

The performance of the proposed framework is evaluated using three metrics.

Actual Wall Clock Time (act_WC) refers to the actual elapsed time required to answer a query, measured from the beginning of the filtering phase to the end of the refinement phase. Total elapsed time per query:

$$\text{act\_WC} = T_{\text{filter}} + T_{\text{refine}}. \tag{6}$$

Finally, we average over 20 random queries:

$$\text{act\_WC} = \frac{1}{20}\sum_{i=1}^{20}\left(T_{\text{filter}}^{(i)} + T_{\text{refine}}^{(i)}\right). \tag{7}$$

The communication cost (act_CC) measures the average size of data exchanged among servers during query processing. The communication cost depends on the output size of the filtering mapper algorithm, and is defined as:

$$\text{act\_CC} = \mathcal{O}(|S_{\text{cand}}| \cdot |t| \cdot |t_j.\text{list}|), \tag{8}$$

where $|S_{\text{cand}}|$ is the number of candidate instances scanned, $|t|$ is the average number of instances per object, and $|t_j.\text{list}|$ is the average count of partially dominated objects. AES compresses these emissions into a single aggregated record containing all competitor identifiers. Aggregated Emission Rate (AER) due to the AES is defined by:

$$\text{AER} = \frac{E_{\text{AES}}}{E_{\text{baseline}}} \times 100\,(\%). \tag{9}$$

D. Results

1) Baseline vs. Proposed Algorithm: The proposed framework integrates *aggregated emission strategy (AES)* to collapse competitor identifiers into a single record per surviving instance, and *dynamic smart candidate pruning (DSCP)* to discard weak objects using LB/UB bounds against a per-partition threshold. For the synthetic dataset, *act_WC* of the baseline averaged $66{,}520\,\text{ms}$, whereas the proposed algorithm achieved $43{,}757\,\text{ms}$ (a 34.2% reduction). Also *act_CC* remained approximately 1.4179×10^{11}. For the real dataset, the baseline *act_WC* averaged $56{,}274\,\text{ms}$, while the proposed algorithm reduced this to $42{,}366\,\text{ms}$ (a 24.7% reduction); communication cost was roughly 1.54×10^{10}. Table II summarizes these improvements. Communication cost remained unchanged as both the baseline and proposed algorithms emitted the same number of records across all datasets. Figure 4 plot wall clock time comparison.

TABLE II
IMPROVEMENT OF PROPOSED ALGORITHM

Dataset	Baseline act_WC	Proposed act_WC	act_CC	Reduction
Synthetic	66,520 ms	43,757 ms	1.4179×10^{11}	34.2%
Real	56,274 ms	42,366 ms	1.543×10^{10}	24.7%

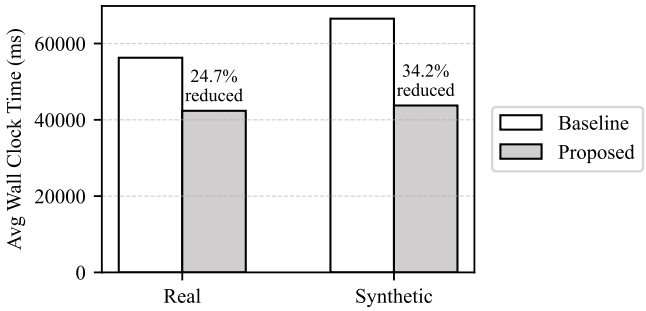

Fig. 4. Baseline vs. Proposed — Average Wall Clock Time (act_WC).

TABLE III
CONTRIBUTION OF AES AND DSCP (WALL CLOCK TIME REDUCTION)

Dataset	AES only	DSCP only	AES+DSCP
Synthetic	32.7%	12.0%	34.2%
Real	23.6%	16.8%	24.7%

2) Ablation Study: To isolate contributions, this work evaluated AES-only and DSCP-only variants. AES accounts for the majority of runtime reduction by shrinking emissions; DSCP yields complementary gains by pruning weak candidates pre-shuffle. On the synthetic dataset, *AES-only* reduced *act_WC* to $44{,}760\,\text{ms}$ ($\approx 32.7\%$ faster than baseline), *DSCP-only* reduced it to $58{,}484\,\text{ms}$ ($\approx 12.0\%$ faster), while the full proposed algorithm achieved the best performance with $43{,}757\,\text{ms}$ ($\approx 34.2\%$ faster). On the real dataset, *AES-only* reduced *act_WC* to $42{,}967\,\text{ms}$ ($\approx 23.6\%$ faster), *DSCP-only* to $46{,}848\,\text{ms}$ ($\approx 16.8\%$ faster), and the proposed algorithm to $42{,}366\,\text{ms}$ ($\approx 24.7\%$ faster). Table III lists the observed reductions, and Figure 5– 6 show the wall-time slices.

VI. CONCLUSION

In this paper, we propose an efficient Distributed framework for Probabilistic Top-k Dominating queries that combines dynamic smart candidate pruning (DSCP) with an aggregated

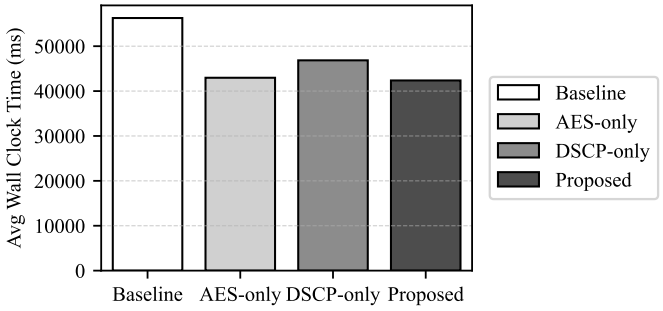

Fig. 5. Ablation Study on Real Dataset.

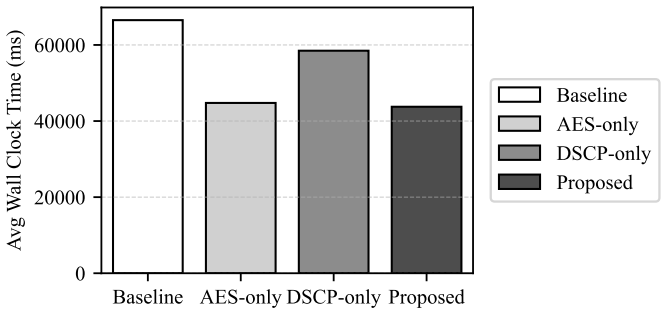

Fig. 6. Ablation Study on Synthetic Dataset.

emission strategy (AES) in Hadoop MapReduce, preserving exactness while reducing wall-clock time by 24–34% on synthetic and real datasets. Using parallel computation in Hadoop and optimized data partitioning strategies, the framework effectively addresses the challenges posed by uncertainty and computational overhead. Experimental results demonstrate that our approach significantly reduces the response time to the query while maintaining high accuracy, scalability, and robustness across diverse real and synthetic datasets. This makes the framework well-suited for real-world applications such as decision support systems, recommendation engines, and sensor data analysis, where uncertainty is inherent. Future work will focus on extending the framework to dynamic and streaming environments, exploring adaptive load-balancing techniques to further enhance efficiency in large-scale distributed settings, and integrating advanced optimization strategies such as machine learning-based pruning for other probabilistic query models. Additionally, implementing the framework on Apache Spark could take advantage of in-memory processing capabilities for improved performance.

REFERENCES

[1] A. Deshpande, C. Guestrin, S. R. Madden, J. M. Hellerstein, and W. Hong, *Model-Driven Data Acquisition in Sensor Networks*. Elsevier, 2004, pp. 588–599.

[2] J. Pei, B. Jiang, X. Lin, and Y. Yuan, "Probabilistic skylines on uncertain data," *VLDB 2007*, pp. 15–16, 2007.

[3] D. Z. Wang, E. Michelakis, M. Garofalakis, and J. M. Hellerstein, "Bayesstore: Managing large, uncertain data repositories with probabilistic graphical models," *Proceedings of the VLDB Endowment*, vol. 1, pp. 340–351, 8 2008.

[4] C.-C. Lai, T.-C. Wang, C.-M. Liu, and L.-C. Wang, "Probabilistic top-k dominating query monitoring over multiple uncertain iot data streams in edge computing environments," 6 2019.

[5] X. Lian and L. Chen, "Top-k dominating queries in uncertain databases," in *Proceedings of the 12th International Conference on Extending Database Technology: Advances in Database Technology*. ACM, 3 2009, pp. 660–671.

[6] H. A. Fattah, K. A. Hasan, and T. Tsuji, "Weighted top-k dominating queries on highly incomplete data," *Information Systems*, vol. 107, p. 102008, 2022.

[7] X. Lian and L. Chen, "Probabilistic top-k dominating queries in uncertain databases," *Information Sciences*, vol. 226, pp. 23–46, 2013.

[8] M. S. Rahman and K. Azharul Hasan, "Computing skyline query on incomplete data," in *International Conference on Big Data, IoT and Machine Learning*. Springer, 2023, pp. 657–672.

[9] J. Dean and S. Ghemawat, "Mapreduce: A flexible data processing tool." *Communications of the ACM*, vol. 53, pp. 72–77, 1 2010.

[10] N. Rai and X. Lian, "Distributed probabilistic top-k dominating queries over uncertain databases," *Knowledge and Information Systems*, vol. 65, pp. 4939–4965, 11 2023.

[11] D. Papadias, Y. Tao, G. Fu, and B. Seeger, "Progressive skyline computation in database systems," *ACM Transactions on Database Systems*, vol. 30, pp. 41–82, 3 2005.

[12] M. L. Yiu and N. Mamoulis, "Efficient processing of top-k dominating queries on multi-dimensional data," *Proceedings of the VLDB Conference*, pp. 483–494, 2007.

[13] X. Han, J. Li, and H. Gao, "Tdep: efficiently processing top-k dominating query on massive data," *Knowledge and Information Systems*, vol. 43, pp. 689–718, 6 2015.

[14] P. Punam, S. Sultana, and K. M. A. Hasan, "A bitmap based algorithm for computing top-k dominating queries," in *2024 27th International Conference on Computer and Information Technology (ICCIT)*. IEEE, 12 2024, pp. 1034–1039.

[15] P. Sun, H. Yu, and Y. Wei, "Top-k query algorithm on massive data," in *2024 IEEE 7th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*. IEEE, 3 2024, pp. 1923–1929.

[16] W. Zhang, X. Lin, Y. Zhang, J. Pei, and W. Wang, "Threshold-based probabilistic top-k dominating queries," *The VLDB Journal*, vol. 19, pp. 283–305, 4 2010.

[17] X. Feng, X. Zhao, Y. Gao, and Y. Zhang, *Probabilistic Top-k Dominating Query over Sliding Windows*, 2013, pp. 782–793.

[18] B. J. Santoso and G.-M. Chiu, "Close dominance graph: An efficient framework for answering continuous top-k dominating queries," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, pp. 1853–1865, 8 2014.

[19] C.-C. Lai, C.-C. Fan, and C.-M. Liu, "An effective pruning scheme for top-k dominating query processing on uncertain data streams," in *2022 IEEE VTS Asia Pacific Wireless Communications Symposium (APWCS)*. IEEE, 8 2022, pp. 104–108.

[20] O. Benjelloun, A. D. Sarma, A. Y. Halevy, and J. Widom, "Uldbs: Databases with uncertainty and lineage," *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB 2006)*, vol. 32, pp. 953–964, 9 2006.

[21] L. Antova, C. Koch, and D. Olteanu, "Maybms: Managing incomplete information with probabilistic world-set decompositions," in *2007 IEEE 23rd International Conference on Data Engineering*. IEEE, 2007, pp. 1479–1480.

[22] F. Li, K. Yi, and J. Jestes, "Ranking distributed probabilistic data," in *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. ACM, 6 2009, pp. 361–374.

[23] Y. Park, J.-K. Min, and K. Shim, "Parallel computation of skyline and reverse skyline queries using mapreduce," *Proceedings of the VLDB Endowment*, vol. 6, pp. 2002–2013, 9 2013.

[24] J. Zhang, X. Jiang, W.-S. Ku, and X. Qin, "Efficient parallel skyline evaluation using mapreduce," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, pp. 1996–2009, 7 2016.

[25] X. Zhou, K. Li, Y. Zhou, and K. Li, "Adaptive processing for distributed skyline queries over uncertain data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, pp. 371–384, 2 2016.